

# Research disclosure — weight recoverability in the shipped UAF.FM.GenerativeModels on-device model assets

Yu Yao-Hsing [euler.yu@gmail.com](mailto:euler.yu@gmail.com) [github.com/yu314-coder](https://github.com/yu314-coder)

To: Apple Product Security

Dear Apple Product Security team,

I am writing to disclose the results of a static reverse-engineering study of the on-device model assets that ship with Apple Intelligence (`com.apple.MobileAsset.UAF.FM.GenerativeModels`). The work was carried out on my own hardware, using only the assets already present on that machine, and I am sending it to you before drawing any further public attention to it.

The full report is linked at the end of this message.

## Why I think this warrants your attention

The shipped assets are sufficient to reconstruct the on-device models' weight tensors. I do not mean that the files are present and opaque — I mean that the storage format is fully reversible, and I have reversed it and checked the result against your own toolchain:

- The weight codec is 4-bit LUT palettization with a per-block fp16 scale, stored under an Apple Neural Engine tile permutation. Both layers are reversible from the shipped bytes alone.
- The parameter budget closes **exactly** against the physical file: 9.862 B parameters, 100.00% of the shipped weight bytes, zero non-finite tensors.
- The decoder is validated by round-trip: compiling a convolution with known random weights through `ANCompiler` and decoding the resulting coefficient stream returns those weights at **correlation 0.981** — the 4-bit quantization floor, i.e. exact up to the quantizer.

Two mechanisms that appear to function as barriers do not, in fact, protect the models:

- **The instruction-following-pruning router→expert map.** This is consumed at export time and does not ship, which makes it look like a hard blocker. It is not one: the pruned feed-forward is an *ungated, permutation-invariant sum* over resident experts, so the router→expert correspondence is mathematically irrelevant to the output. Withholding the map does not withhold the model's behaviour.
- **The “missing constant table”** referenced by `config.json` but absent from the package has a functionally equivalent form in the shipped `metadata.bin` swizzler tables.

## The item I would most like you to look at

Between the asset catalogue I first analysed and the one on macOS 27.0 build **26A5378n**, the catalogue grew from 118 to 124 assets, and one of the new assets is:

`UC_FM_LANGUAGE_INSTRUCT_3B_EMBEDDINGS_GENERIC_GENERIC_H16G_IFP_Cryptex.dmg`

It contains a 268,442,456-byte file whose size is exactly  $262144 \times 2048$  four-bit values plus a 7000-byte header, with `metadata.json` reading `{"vocab_size": 262144, "embedding_dim": 2048}` and an odix header naming `$load_embeddings`.

This matters because the *absence* of that embedding in earlier builds was the single reason a standalone reconstruction was not possible. I had verified that absence carefully: roughly 1.14 M candidate offsets across the entire weight file scored  $\leq +0.047$  against a probe calibrated at +0.53 on a genuine embedding, and an 8.2 GB full-memory core taken during inference contained zero buffers of the required width, because that model's gather/dequantize runs on the ANE and never crosses into host memory.

A later build now ships that table as a static, mountable, read-only cryptex requiring no privilege to open. If the earlier posture was deliberate, this looks like an unintended regression, and it is the one item here I would flag as time-sensitive.

For completeness: I have **not** recovered per-token vectors from that file. The row stride and an eight-lane storage structure are established; the element layout is not. I am reporting the exposure, not a completed extraction.

## What still does protect the models

I want to be even-handed, because two things genuinely held:

- **ANE coefficient ordering.** Projections that write into the residual stream are emitted with `OutTrans=1`, a storage transpose selected by the ANE scheduler from whole-graph context. No compiler I could drive emits it for an isolated operator, and it is the sole reason my reconstruction does not produce coherent output. Five of seven weight roles per layer decode correctly; two do not.
- **Activation opacity.** Intermediate activations never leave the ANE. I verified this by scanning every aligned offset of an 8.2 GB process core and finding no residual-stream buffer. This removes any per-layer ground truth and is a real barrier to validation.

## What I did not do

No signing material was bypassed and no protection was circumvented. The static analysis reads the shipped, already-unencrypted Cryptex images. The single dynamic step is a user-privileged `lldb` core dump of my own process on my own machine, used to observe a buffer that the CPU places in host memory during normal operation.

I have not published, distributed, or uploaded any Apple weights, tokenizer data, or model assets, and I do not intend to. The public repository accompanying this work contains only my own analysis and decoder source; a filter blocks weight and asset file types from it. The linked folder is subject to the same rule — it holds the report and my own code, and no Apple content.

If you would like the reproduction artefacts, I will provide them through whatever channel you prefer rather than over a shared link.

## **Report**

**[ paste the Google Drive link here before sending ]**

The folder contains the full technical report (PDF, 23 pages) and the decoder source. The report documents the recovery pipeline in full, including the parts that failed: four of my own intermediate results are retracted in place, each traceable to a control that destroyed more structure than the hypothesis it was testing.

I am happy to answer questions, to walk through any part of the method, or to delay further publication while you assess this. I have no timeline of my own and am not seeking compensation; I would simply rather you saw this first.

Kind regards,

**Yu Yao-Hsing** euler.yu@gmail.com github.com/yu314-coder